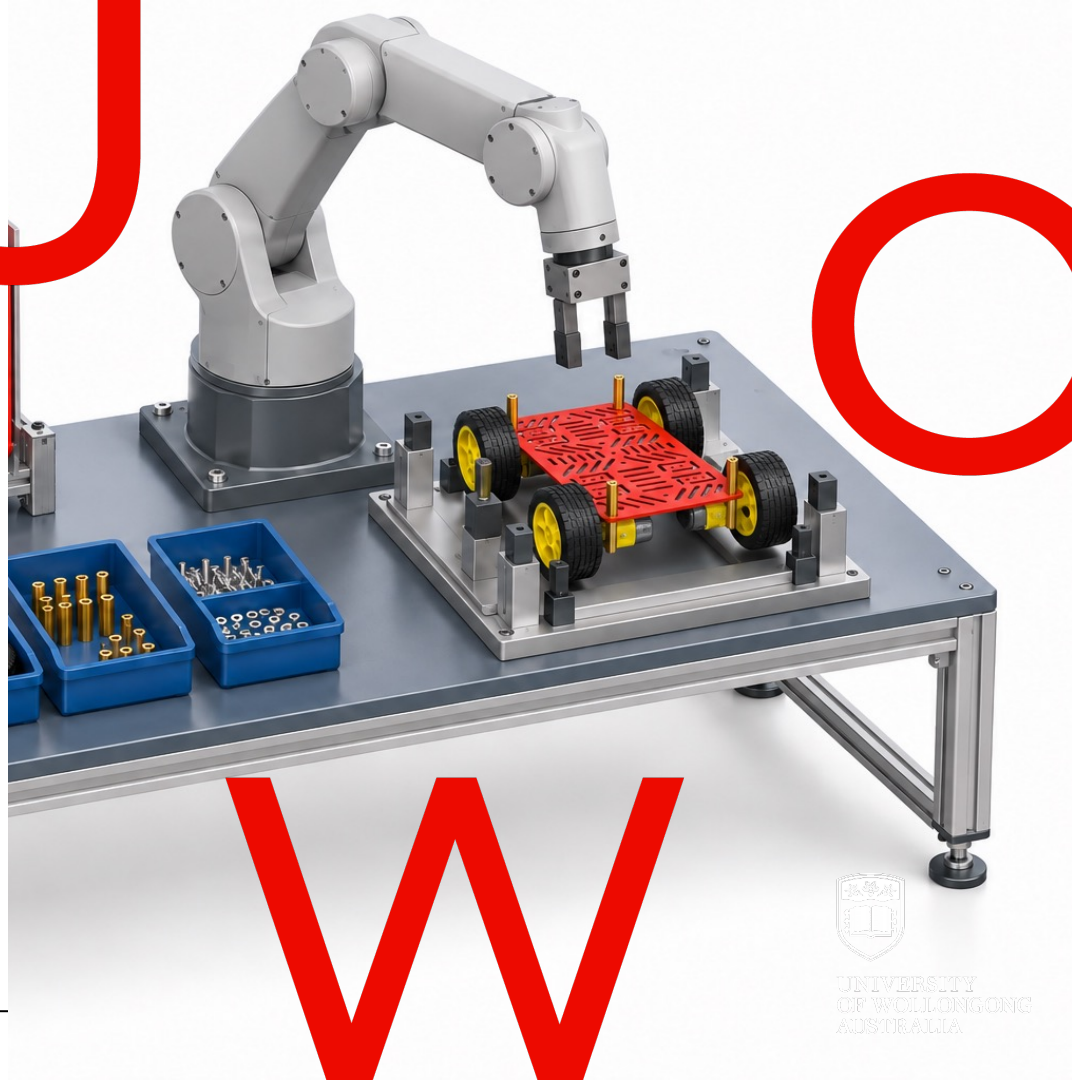
Robotics and Flexible Automation

Lecture 6

ABB RAPID programming

Prof. Zengxi (Stephen) Pan

BJTU· 2026



Robot Programming: What Must Be Defined

CORE

Every programming method must produce complete executable information

Programming information	What must be defined
Target pose and configuration	`Cartesian pose plus a valid robot configuration`
Tool and payload	`TCP, tool orientation, mass and centre of gravity`
Workobject and reference frames	`The object frame relative to the robot base`
Motion	`Joint, linear or circular motion with speed and blending`
Process sequence and controller I/O	Operation order, conditions, signals and process commands

Methods for Robot Programming

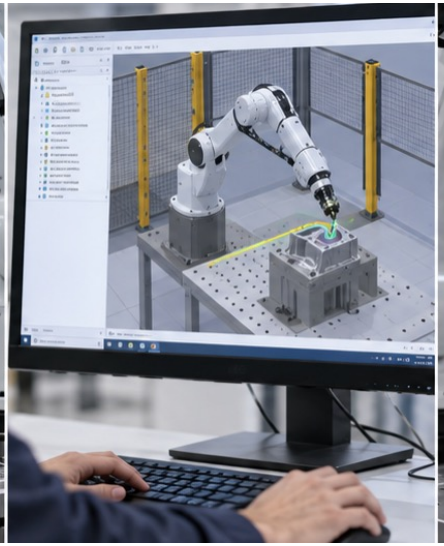
CORE

Direct teaching, offline models, sensor feedback and automatic program generation



Online teaching

Jog and store targets on the real robot



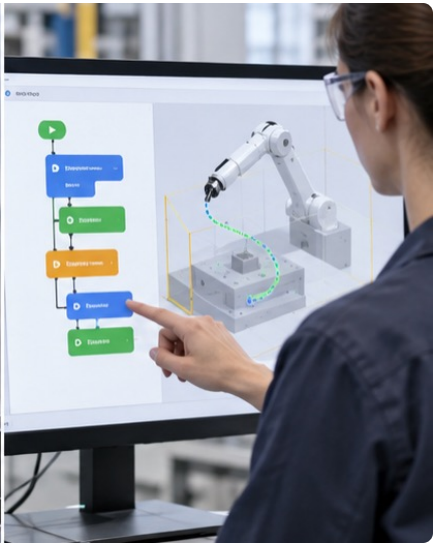
Offline programming

Build and test the cell in a virtual controller



Sensor-guided motion

Vision or force feedback corrects the path



Automatic / no-code

Software generates RAPID and collision-free motion

Online Programming

CORE

Teach targets and edit the program on the real robot

How it works	Jog the physical robot with the FlexPendant, record target poses, then edit and test the controller program.
Benefits	Directly verifies reach, configuration, tooling and the real cell. Effective for commissioning and touch-up.
Limitations	Occupies the robot and cell. Point-by-point teaching becomes slow for dense paths, and the operator may need access to the safeguarded area.



ABB OmniCore FlexPendant

Offline Programming

CORE

Build, program and verify the robot cell in simulation

How it works	Import the robot, tool, fixture and CAD data. Create targets and RAPID, simulate motion, I/O and collisions, then synchronise with the controller.
Benefits	Programming does not stop production. Reach, paths, cycle time and collisions can be checked before commissioning.
Limitations	The virtual cell must match the real cell. Tool, workobject and base calibration errors still require commissioning and touch-up.

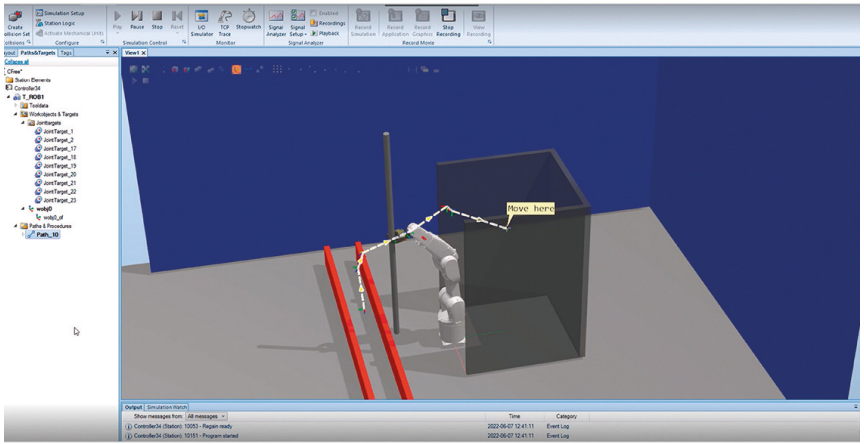


ABB RobotStudio and the ABB Virtual Controller

Sensor-Based Programming

CORE

Sensor feedback changes targets or paths during execution

How it works	Vision, force or scan data localises the part or process. The controller applies measured corrections while the program runs.
Benefits	Compensates for part variation and supports localisation, tracking and contact tasks that fixed taught points cannot handle reliably.
Limitations	Accuracy depends on calibration, sensor quality and latency. Corrections must stay inside validated motion and safety limits.

Play video

Sensor-based programming

From the course website

ABB Integrated Vision: camera, controller, robot and RobotStudio

Open video on the course website

Automated and No-Code Programming

CORE

Software generates robot motion and native controller code from task data

How it works	Import the CAD model, identify the welds, and define process requirements. The software plans collision-free robot motion and exports native robot code.
Benefits	Reduces point-by-point programming for high-mix welding. The user specifies the task while the software handles path and configuration planning.
Limitations	The workflow is application-specific and depends on an accurate cell model. The engineer still validates reach, calibration, process settings, safety and generated code.

Play video

Automated programming

From the course website

Verbotics Weld: CAD-based automatic robot programming

Open video on the course website

RAPID: ABB's Robot Programming Language

CORE

Introduced with the S4 controller in 1994 and retained across ABB controller generations

RAPID runs on ABB RobotWare controllers and controls robot motion, process logic and I/O.

Controller history

S4 and RAPID (1994)
IRC5 and RobotWare (from 2004)
OmniCore and RobotWare 7

Program structure

Modules contain declarations and routines.
Routines are procedures, functions or traps.
The controller interprets the loaded program.

```
MODULE testPrg

  PROC main()
    MoveJ p10, v200, z50, tool1;
    MoveL p20, v100, fine, tool1;
  ENDPROC

ENDMODULE
```

The same language is edited online on the controller and offline in RobotStudio.

Example: RAPID Program Structure

EXAMPLE

The declarations define the robot data; main() executes three motion instructions

```
MODULE DeburrModule

  CONST robtarget p10 := ...;
  PERS tooldata tDeburr := ...;

  PROC main()
    MoveJ p10, v200, z50, tDeburr;
    MoveL p20, v100, fine, tDeburr;
    MoveL p30, v100, fine, tDeburr;
  ENDPROC

ENDMODULE
```

What this program does

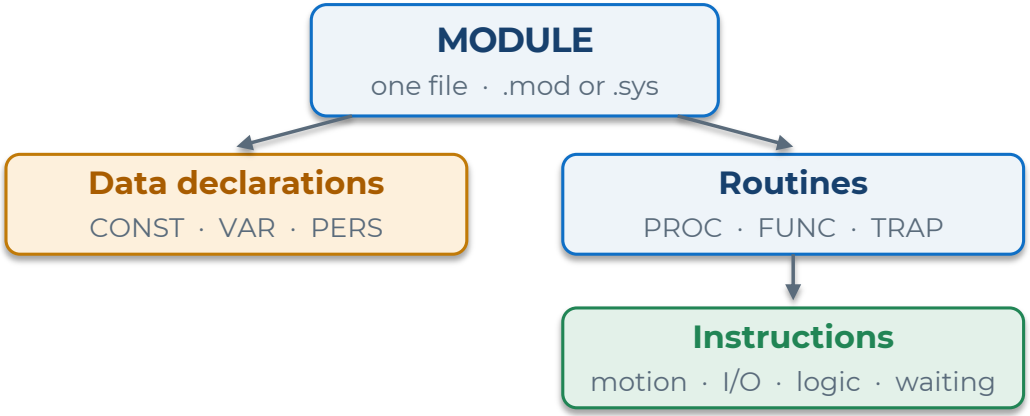
1 Stores p10 as a robtarget.
2 Defines tDeburr, including its TCP and load.
3 MoveJ approaches p10.
4 MoveL moves through p20 and p30.
5 fine requires the TCP to stop at p20 and p30.

Template only: replace ... with data and declare p20/p30 before execution.

Modules, Components and Files

CORE

What a program is made of, and what is stored on disk



Data says what the motion uses.
Instructions say what happens.

File	What it holds
.mod	a program module — loaded and unloaded during execution
.sys	a system module — common data and routines, normally always available
.pgf	the program file that references the modules making up one program

Loading a program replaces the program modules it references. System modules are left alone.

Three Kinds of Routine

CORE

What the routine returns decides how it can be used

PROC — procedure

A set of instructions that returns nothing.

Called as a subprogram: PickWheel;

FUNC — function

Returns a value of a declared type.

Used inside an expression: Offs(p10,0,0,50)

TRAP — trap routine

Runs in response to an interrupt.

Connected to an event, then triggered by it.

Almost all project code is procedures. Functions and traps earn their place only when the same calculation or the same event recurs.

Three Kinds of Data

CORE

The declaration decides when a value may change, and whether it survives a restart

```
CONST num nMaxSpeed := 500;
VAR   num nPartCount := 0;
PERS  num nTotalCount := 0;
```

Kind	Behaviour
CONST	a fixed value; changed only by editing the declaration
VAR	may change while the program runs; re-initialised when the program is reset
PERS	keeps its latest value — the declaration is rewritten as the program runs

Targets and tools are usually CONST or PERS. A counter is a VAR. Getting this wrong is why a taught target quietly reverts.

Anatomy of a Move Instruction

CORE

Where to go, how fast, how accurately, and which point moves

```
MoveL  p10  v200  z20  tool1  \WObj:=wobj1
```

- **MoveL** — which interpolation
- **p10** — robtarget — where to go
- **v200** — speeddata — how fast
- **z20** — zonedata — how accurately
- **tool1** — tooldata — which point moves
- **\WObj:=wobj1** — wobjdata — in which frame

Change any one argument and the path, the timing, the accuracy or the physical point being moved changes with it.

The controller supplies

the interpolation, the servo tracking and the corner it actually cuts.

You must declare

the target, the speed, the zone, the tool and the work object — and why each is right.

MoveJ: Joint-Interpolated Motion

CORE

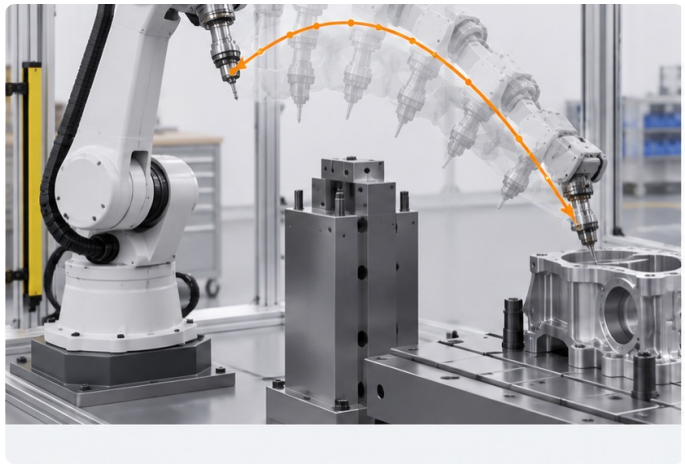
Each joint is interpolated; the TCP path follows from the robot kinematics

```
MoveJ p10, v200, z50, tool1 \WObj:=wobj1;
```

Fast and robust: no Cartesian singularity and no reach problem between the two targets can stop it.

The TCP sweeps a curve nobody drew — clearance must be checked over the whole swept volume.

The right default for free-space transfer.



Use MoveJ for free-space transfer. Verify that the swept volume is collision-free.

MoveL: Cartesian Linear Motion

CORE

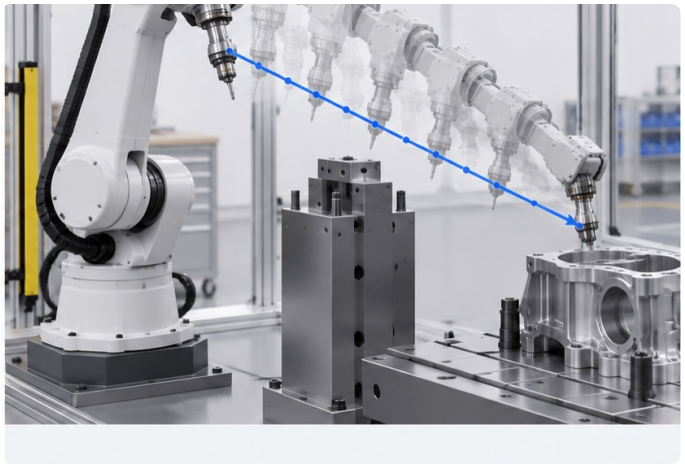
The TCP follows a straight line in the active workobject frame

```
MoveL p20, v100, fine, tool1 \WObj:=wobj1;
```

IK is solved at every interpolated pose, all on one branch.

Reachable endpoints do not prove the line between them.

"Linear" means linear in the named frame.



MoveL can fail during the segment at a singularity, joint limit or workspace boundary.

MoveC — Circular Movement

CORE

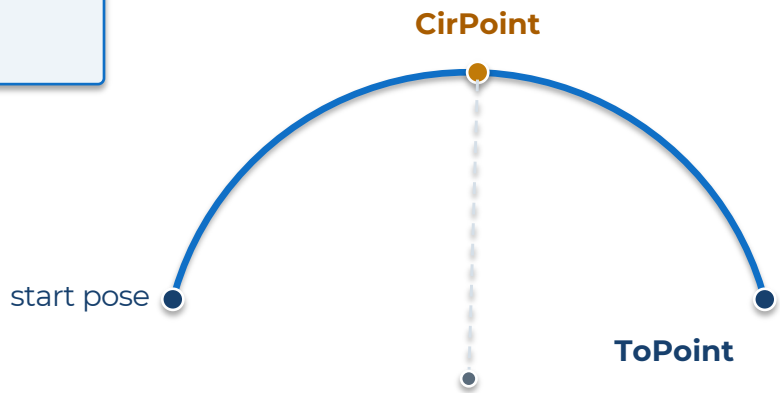
Three poses fix the arc: where the tool is, a point on the path, and the destination

```
MoveC pCirc, pEnd, v200, z10, tool1 \WObj:=wobj1;
```

The first robtarget is the CirPoint — a point on the path, not a corner, not the centre.

The second is the destination.

Nearly collinear points define the circle badly: refused, or a huge arc.



the arc is fixed by three poses — the CirPoint lies on the path, not at the centre

Use MoveC when the process needs an arc. Rounding a corner is a job for zone data, not for a circle.

Choosing the Instruction

CORE

The instruction encodes process intent, not merely a different syntax

Instruction	What is constrained	Use it for	What you must verify
MoveJ	joint interpolation	free-space transfer, large reconfiguration	the swept volume of robot, tool and carried part
MoveL	a straight TCP path	approach, retreat, insertion, dispensing	IK continuity, singularity, joint speed
MoveC	a circular TCP path	arcs and contours	CirPoint geometry and orientation along the arc

A workable rule: MoveL wherever the tool is near the work, MoveJ everywhere else — then justify every exception.

speeddata — How Fast

CORE

Four components: the TCP, the reorientation and two external axis rates

```
VAR speeddata vSlow := [ 20, 30, 200, 15 ];
                    !  v_tcp v_ori v_leax v_reax

MoveL p20, vSlow, fine, tool1 \WObj:=wobj1;
```

vtcp — TCP velocity, mm/s.

vori — tool reorientation, degrees/s.

vleax, vreax — external axes.

Predefined v5 … v7000 set vtcp to that many mm/s.

A large reorientation at a small vtcp can still be the slowest part of a move — vori is the component people forget.

zonedata — How Accurately

CORE

How close the robot must come before it starts the next segment

```
VAR zonedata zMy :=
    [ FALSE, 10, 15, 15, 1.5, 15, 1.5 ];
  !   finep  pzone_tcp pzone_ori pzone_eax
  !          zone_ori  zone_leax zone_reax

MoveL p20, v200, zMy, tool1;
```

finep TRUE makes a stop point; every other field is ignored.

pzonetcp is the TCP zone radius, in mm.

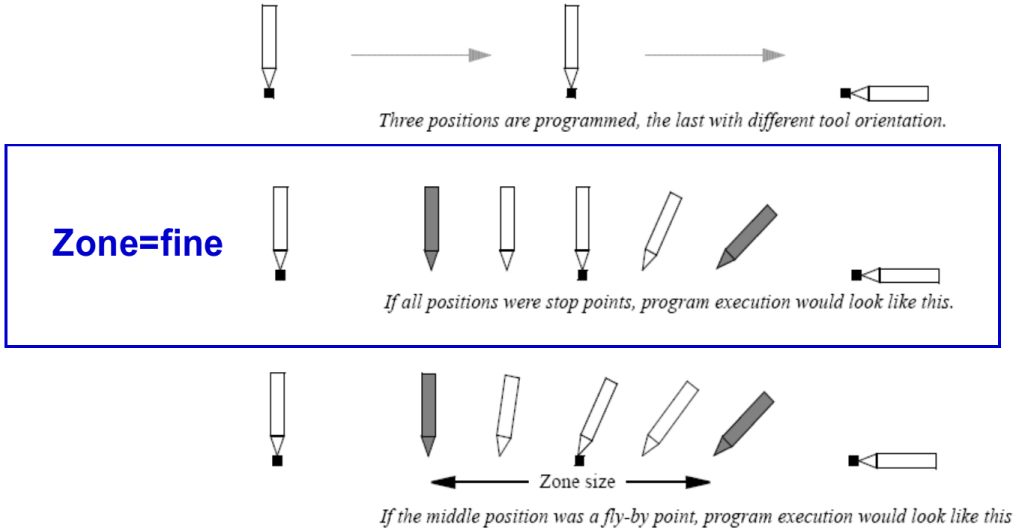
The rest size the larger zone used for reorientation and external axes.

fine and the predefined z1 … z200 cover almost every case. Declare your own zonedata only when they do not.

Stop Points and Fly-by Points

CORE

A stop point is reached; a fly-by point never is



The zone applies to the tool orientation as well as to the TCP — the tool starts turning before the corner.

Inside a Zone

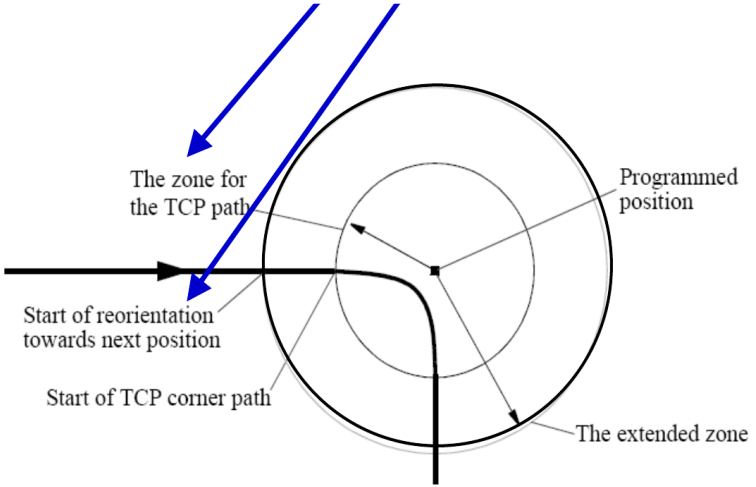
CORE

The programmed position is never attained — by design

The controller leaves the programmed path at the zone boundary and rejoins the next segment beyond it.

The deviation is bounded by the zone radius; the blended curve itself varies with speed.

A zone on a process target is a defect — the tool misses the point it was told to reach.



Fine where the tool touches the work. Zones only on the free-space moves between.

Predefined Zone Values

CORE

The name is the TCP zone radius in millimetres

Name	TCP path	Reorientation	Typical use
fine	0 mm — stop point	—	contact, insertion, measurement
z1	1 mm	1 mm / 0.1°	a corner that must stay sharp
z5	5 mm	8 mm / 0.8°	short approach and retreat moves
z10	10 mm	15 mm / 1.5°	general process-adjacent motion
z50	50 mm	75 mm / 7.5°	free-space transfer
z200	200 mm	300 mm / 30°	long transfer, cycle time critical

The full table runs z0 to z200 with six numeric columns — take the exact figures from the manual, not from a slide.

Choosing Speed and Zone

CORE

Two regimes, and the programme should make clear which one each line is in

Transfer, approach and retreat

High speed — typically above 150 mm/s.

Large zones, z5 and above, so the speed never has to collapse at a corner.

Process points and contact

fine: reach the point and stop (grip/release).
Non-zero zone: blend only within process tolerance.

Use the highest speed the process and the verification support, and avoid abrupt changes between segments.

Cycle time is won on the transfer moves, not by rushing the process points.

Example: A Rectangular Path with Stop Points

EXAMPLE

Four targets, five instructions, and a full stop at every corner

```
PROC main()
  MoveL p10, v200, fine, tPen;
  MoveL p20, v200, fine, tPen;
  MoveL p30, v200, fine, tPen;
  MoveL p40, v200, fine, tPen;
  MoveL p10, v200, fine, tPen;
ENDPROC
```



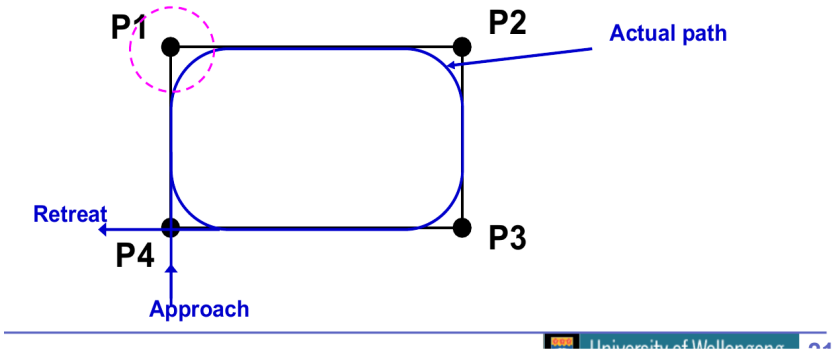
The path is exactly the programmed rectangle. It is also the slowest way to draw one.

Example: The Same Path with Zones

EXAMPLE

Only the zone argument changes — the targets are untouched

```
PROC main()
  MoveL p10, v200, fine, tPen;
  MoveL p20, v200, z20,  tPen;
  MoveL p30, v200, fine, tPen;
  MoveL p40, v200, z50,  tPen;
  MoveL p10, v200, fine, tPen;
ENDPROC
```



The robot rounds p20 and p40 and never reaches them. It still stops exactly on p10 and p30.

Path and Target

CORE

A path is a sequence of move instructions addressed to targets

Position

Where the TCP must be, expressed in the active work object frame.

Orientation

How the tool must be turned when it arrives — a unit quaternion.

Configuration

Which inverse-kinematic solution the controller must use to get there.

External axes

Track or positioner coordinates, where the cell has them.

A target is not a point. It is a complete statement of how the robot must stand when the tool gets there.

robtarget — The Four Fields

CORE

Each field answers a different execution question

trans	`[ x, y, z ]`	where the TCP goes, in the active work object
rot	`[ q1, q2, q3, q4 ]`	how the tool is oriented there — a unit quaternion
robconf	`[ cf1, cf4, cf6, cfx ]`	which inverse-kinematic branch the controller must use
extax	`[ eax_a … eax_f ]`	external axes; 9E9 marks an axis that is not used

```
CONST robtarget p10 := [ [x,y,z], [q1,q2,q3,q4], [cf1,cf4,cf6,cfx], [eax_a ... eax_f] ];
```

Position and orientation are relative to the work object. Change the work object and the same numbers mean a different place.

Example: Reading a robtarget

EXAMPLE

One declaration, four separate statements about the robot

```
CONST robtarget p15 := [
    [600, 500, 225.3],
    [1, 0, 0, 0],
    [1, 1, 0, 0],
    [9E9,9E9,9E9,9E9,9E9,9E9] ];
```

The TCP goes to x = 600, y = 500, z = 225.3 mm in the object frame.

The tool is aligned with the object frame — [1,0,0,0] is the identity quaternion.

Axes 1 and 4 must be in quadrant 1, that is 90° to 180°; axis 6 in quadrant 0, that is 0° to 90°.

No external axes are used.

Read a robtarget aloud in this order — position, orientation, configuration — and the faults become obvious.

Orientation as a Unit Quaternion

CORE

Four numbers, one orientation — the form every ABB target uses

```
rot = [ q1, q2, q3, q4 ]

identity  = [1, 0, 0, 0]
180° about x = [0, 1, 0, 0]
```

More compact than a 3 × 3 matrix; axis and angle read out of it.

No representation singularity — which is why the controller uses it.

Use the Lecture 2 conversions for a target from Euler angles.

Teaching on the robot fills the four numbers for you.

You will rarely type a quaternion. You will often need to check that the one in front of you means what you think.

Robot Configurations for One TCP Pose

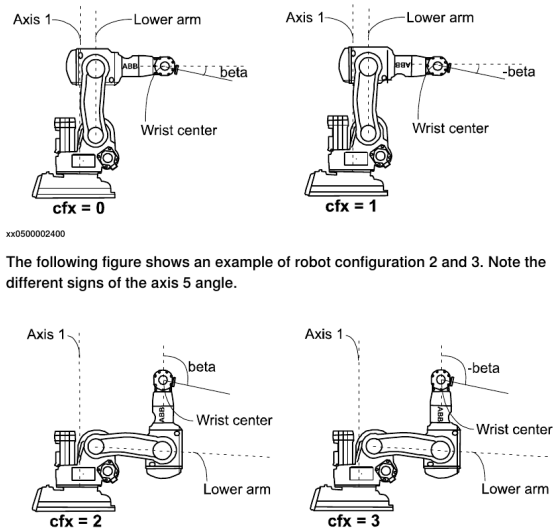
CORE

ABB configuration data selects the intended inverse-kinematic branch

A Cartesian pose fixes the TCP position and orientation. It does not uniquely fix all six joint angles.

For a standard six-axis arm, shoulder, elbow and wrist choices can produce several valid configurations.

confdata records the selected branch so the controller reproduces the taught posture.



Same TCP pose; cfx = 0, 1, 2 and 3 use different wrist and lower-arm postures

A valid endpoint pose is not enough. The program must also select a feasible robot configuration for the complete path.

confdata and Configuration Supervision

CORE

Quadrant data identifies the branch; ConfL and ConfJ control whether it must be preserved

90°

```
confdata = [cf1, cf4, cf6, cfx]
```

Axis quadrant	Angle range
0	0° to 90°
1	90° to 180°
-2	-180° to -90°
-1	-90° to 0°

cf1, cf4 and cf6 store the quadrants of axes 1, 4 and 6. cfx selects the overall robot configuration for the robot type.

Configuration supervision

```
ConfL \On;   ! keep stored configuration for linear
motion
ConfL \Off;  ! allow another feasible configuration

ConfJ \On;   ! supervise joint-motion configuration
ConfJ \Off;  ! allow the closest feasible joint
solution
```

Commissioning rule

Leave supervision on unless a deliberate branch change is required. A configuration fault often reveals an unreachable pose or an unsafe path transition.

tooldata — Which Point Moves

CORE

The TCP frame on the wrist, and the load the robot is carrying

```
PERS tooldata gripper := [
   TRUE,                          ! robhold
   [[97.4,0,223.1],[0.924,0,0.383,0]],   ! tframe
   [5,[23,0,75],[1,0,0,0],0,0,0] ];      ! tload
```

robhold — TRUE when the robot holds the tool.

tframe — TCP position and orientation relative to the wrist.

tload — mass, centre of gravity and inertia of what is carried.

The controller uses this for

the dynamics: with a correct tload it can plan accelerations that the arm can actually deliver.

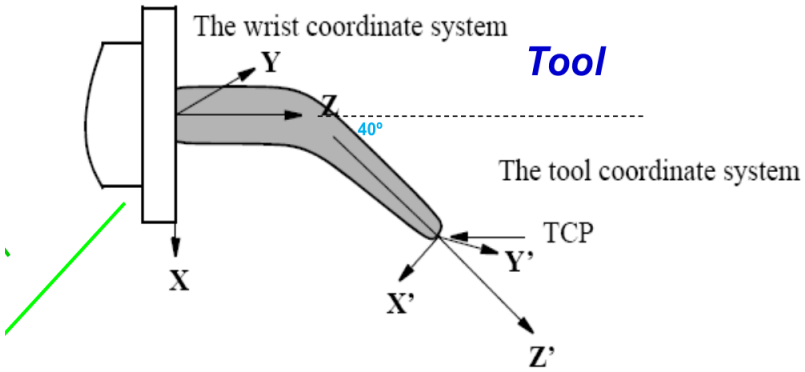
You must supply

the real tool geometry and the real mass — including the part while it is being carried.

Example: The Tool Frame of a Torch

EXAMPLE

What tframe has to describe for a bent tool



The wrist frame is fixed by the robot.

tframe is the offset to the point you care about.

A 40° bend means tool z is not wrist z.

Get it wrong and every target is wrong alike.

Calibrate the TCP on the real robot before trusting any taught or imported target.

wobjdata — In Which Frame

CORE

Two frames: where the fixture stands, and where the part sits on it

```
PERS wobjdata wobj2 := [
   FALSE, TRUE, "",              ! robhold, ufprog, mec
   [[300,600,200],[1,0,0,0]],    ! uframe
   [[0,200,30],  [1,0,0,0]] ];   ! oframe
```

uframe locates the fixture in world coordinates.

oframe locates the object frame inside the user frame.

Every robtarget is interpreted in the active object frame.

Move the fixture, re-measure one frame, and the whole programme still runs. That is what work objects are for.

The Coordinate-System Chain

CORE

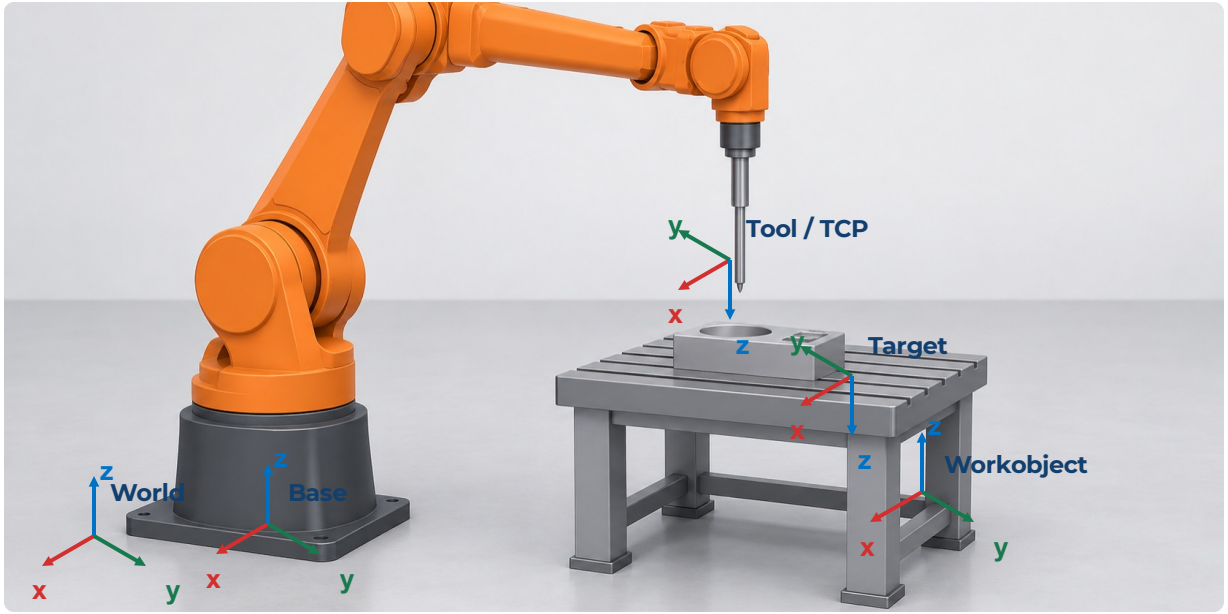
World and base locate the robot; tool Z follows the physical tool axis

Target chain

World and base locate the robot.
The workobject locates the fixture.
The target pose is expressed in that workobject.

Tool chain

Tool Z points from the flange through the TCP. With this downward-facing tool, +Z is downward.



`World / Base → Workobject → Target | Flange → Tool / TCP`

Example: One Target in Two Workobjects

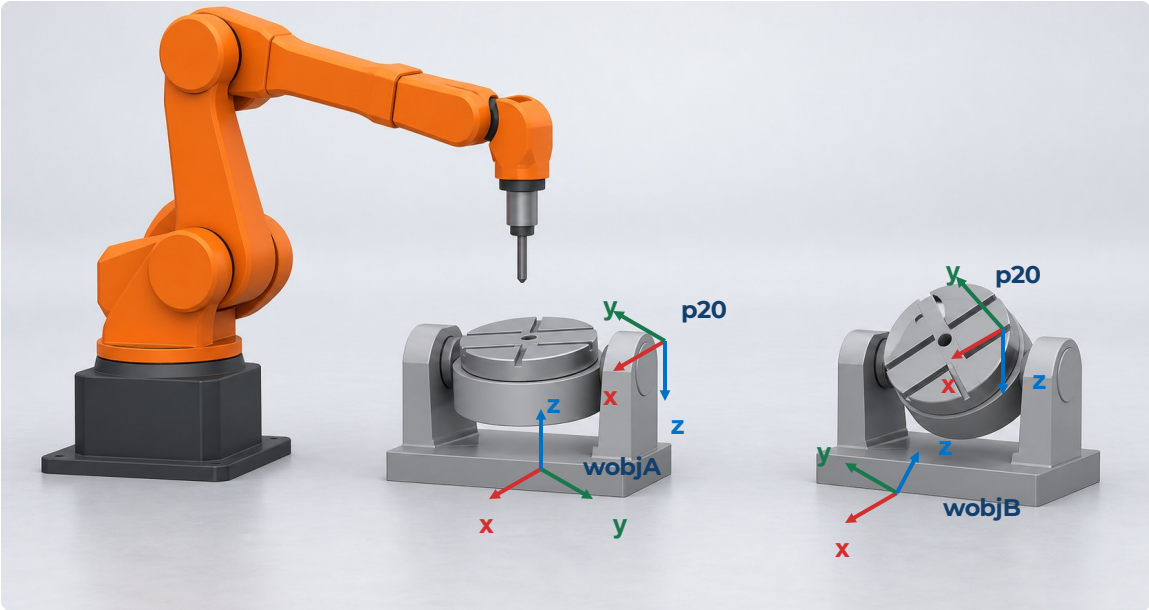
EXAMPLE

The local target frame moves and rotates with its fixture

```
MoveL p20, v100, fine, tGrip
\WObj:=wobjA;

MoveL p20, v100, fine, tGrip
\WObj:=wobjB;
```

p20 has the same local position and orientation in both instructions. The active workobject places and orients it at a different fixture.



The target Z-axis follows the local fixture orientation; it is not automatically the world-up direction.

Geometric Problems with MoveL

CORE

Reachable endpoints do not guarantee a valid Cartesian motion

Intermediate points unreachable

The straight segment leaves the working envelope even though both targets are inside it.

High joint rates near a singularity

A modest TCP speed demands axis speeds the robot cannot deliver.

Endpoints in different configurations

The two targets need different inverse-kinematic branches, and a linear move cannot switch.

All three appear between the targets, never at them — which is why teaching two good points proves nothing.

Intermediate Points Unreachable

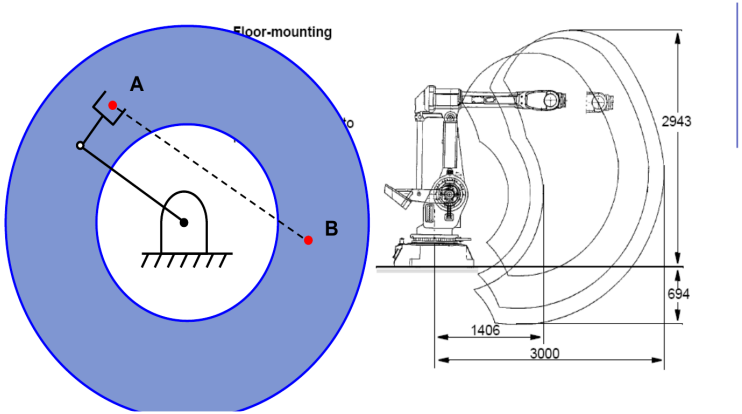
CORE

Both targets solve; the straight line between them does not

The fault is reported part-way through the move, with the tool already committed.

The fix is geometric: add a via point, or use MoveJ where the straight path was never required.

Check the segment in simulation before running it, not after the error.

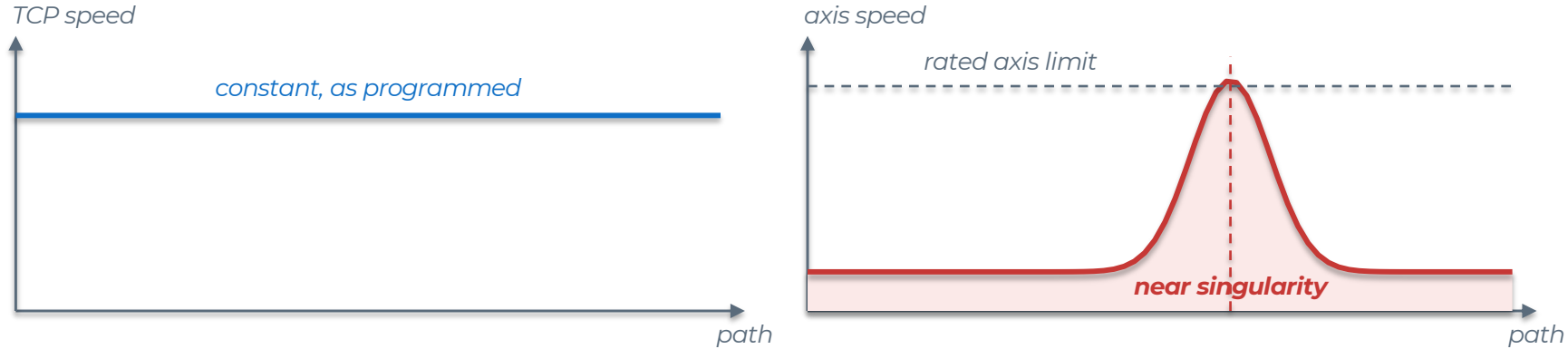


A via point costs one line. A collision costs a gripper.

High Joint Rates Near a Singularity

CORE

The TCP speed is modest; the axis speed required to deliver it is not



Near a wrist singularity axes 4 and 6 nearly align; holding the TCP on the line needs very large rates.

The symptom is a speed-limit fault or a slowdown, not a reachability error.

SingArea lets the controller deviate slightly — use it deliberately, not by default.

Endpoints in Different Configurations

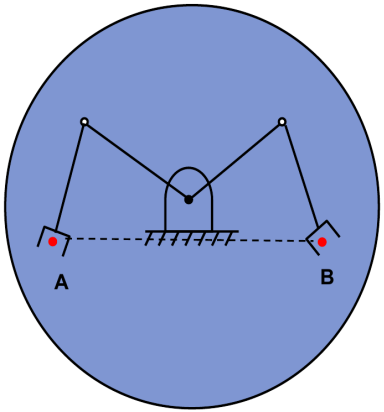
CORE

A linear move cannot change inverse-kinematic branch on the way

Both targets are reachable and both were taught — the stored configuration differs.

With ConfL on the controller refuses the move rather than arriving in the wrong posture.

Re-teach one target in the other branch, or use a MoveJ.



Keep one configuration for a whole process sequence, and change it only in free space.

Beyond Motion: I/O and Waiting

CORE

A programme that only moves cannot pick anything up

```
Set    doGripClose;      ! digital output to 1
Reset  doGripClose;      ! digital output to 0
SetDO  doVacuum, 1;      ! output to a stated value

WaitDI diPartPresent, 1; ! block until the input is 1
WaitTime 0.4;            ! block for a fixed time
```

Gripper actions use digital outputs; the project permits signal placeholders.

Wait for a sensor where one exists; for a time only where there is none.

A move with a zone does not stop — put a fine stop before I/O that needs a precise pose.

Signal names are cell-specific. Declare them once, comment them, and never hard-code a number.

Wheel Insertion: RAPID Program Structure

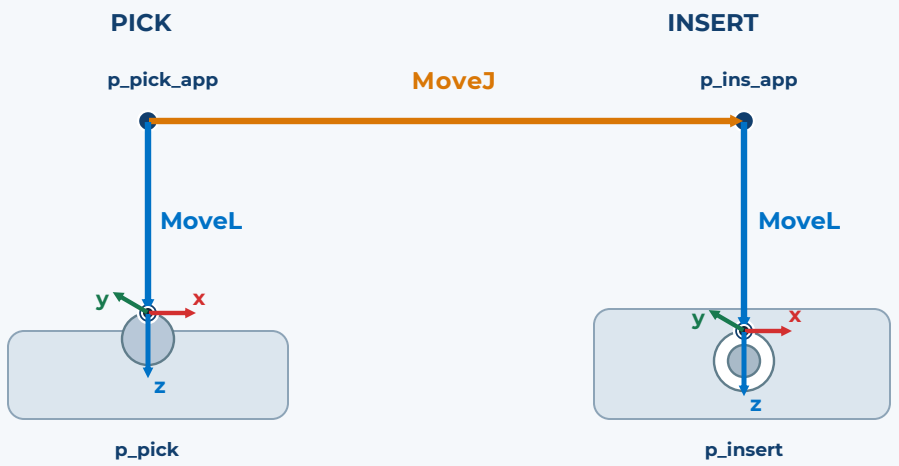
EXAMPLE

Vertical schematic: orient insertion along the actual motor shaft

```
MODULE WheelCell
 PERS tooldata tGrip := [TRUE,
   [[0,0,145],[1,0,0,0]],
   [1.8,[0,0,60],[1,0,0,0],0,0,0]];

 PERS wobjdata wobjPanel := [FALSE,TRUE,"",
   [[520,180,0],[1,0,0,0]],
   [[0,0,0],[1,0,0,0]]];

 PROC main()
   ConfL \On;
   MoveJ pHome, v500, z50, tGrip;
   PickWheel;
   InsertWheel;
   MoveJ pHome, v500, fine, tGrip;
 ENDPROC
```



Pick +Z points down; insertion +Z follows the motor shaft.

MoveJ approach → MoveL pick → retreat → MoveJ insertion approach → MoveL insert → retreat → home

Example: Routines, Approach and Retreat

EXAMPLE

The same five-target pattern, written twice and reused

```
PROC PickWheel()
  MoveJ pPickApp, v500, z20,  tGrip \WObj:=wobjSupply;
  MoveL pPick,    v100, fine, tGrip \WObj:=wobjSupply;
  Set doGripClose;  WaitTime 0.4;
  MoveL pPickApp, v100, z20,  tGrip \WObj:=wobjSupply;
ENDPROC
```

```
PROC InsertWheel()
  MoveJ pInsApp,  v500, z20,  tGrip \WObj:=wobjPanel;
  MoveL pInsert,  v30,  fine, tGrip \WObj:=wobjPanel;
  Reset doGripClose;  WaitTime 0.4;
  MoveL pInsApp,  v100, z20,  tGrip \WObj:=wobjPanel;
ENDPROC
```

MoveJ to the approach pose — generous zone.

MoveL along the insertion axis — slow, fine stop.

Insertion runs at v30: a contact operation.

Retreat before any transfer begins.

Same pattern, different data.

Common Mistakes

OPTIONAL

Symptom	Usual cause
The tool reaches the wrong point	tooldata not calibrated, or the wrong tool named in the instruction
Everything is offset by the same amount	the work object is wrong, or the target was taught in wobj0
The robot misses a pick position	a zone on a process target — it should be fine
Configuration fault part-way through a move	the two targets were taught in different branches
Speed-limit fault mid-path	the linear path passes close to a singularity
A taught target reverts after a reload	the declaration is CONST in a module that was replaced on load

Summary and Project Checkpoint

CORE

RAPID is where the motion plan becomes something a machine will do

Declare
tool, work object and targets

Command
MoveJ, MoveL, MoveC with speed and zone

Structure
routines for each operation

Verify
in simulation, then on the controller

Checkpoint: bring a RAPID programme for one complete operation — declarations, approach, process, retreat, and the I/O placeholders.

Be ready to say, for every argument on every move instruction, why it is that value and not another.

Next: the Jacobian — why the joint speeds behave as they do along the paths you have just written.


UNIVERSITY
OF WOLLONGONG
AUSTRALIA
Stand
for purpose
THANK YOU!